

详细答案(解析版)

1-决策树回顾

1.

(a) 2

不能忽视 x_1 和 x_2 可能为小数的情况。对于这种情况，我们需要考虑使用floor和ceil函数。floor函数可以返回不大于输入参数的最大整数，ceil函数则返回不小于输入参数的最小整数。

我们的目标是分类黑色和白色区域，其中黑色区域被标记为+1，白色区域被标记为-1。根据题目给出的情况，我们知道一个点的标签取决于它所处的区域，如果在边界或角上，那么它的标签和它上方和/或右方的区域相同。因此，我们可以使用ceil函数来对 x_1 和 x_2 进行处理。

- 在节点A，我们检查 $\text{floor}(x_1) \bmod 4$ 的值，如果值为0或2，我们移动到节点B，否则我们移动到节点C。
- 在节点B，我们检查 $\text{floor}(x_2) \bmod 4$ 的值，如果值为0或2，我们将其标记为-1（对应白色），否则我们将其标记为+1（对应黑色）。
- 在节点C，我们也检查 $\text{floor}(x_2) \bmod 4$ 的值，如果值为1或3，我们将其标记为-1（对应白色），否则我们将其标记为+1（对应黑色）。

基于以上描述，我们需要的二叉决策树至少需要有2层。第一层根据 $\text{floor}(x_1)$ 是否是0或2进行判断，第二层根据 x_2 的ceil值是否是1或3进行判断。

为什么会用floor/ceil:

在这个问题中，我们正在试图创建一个二叉决策树来分类一个规律的棋盘图案。棋盘图案是周期性的，即在每个单位长度（或者说每个“棋盘格”）上，颜色是有规律变化的。这个规律是基于棋盘格的位置，即它的 x_1 和 x_2 坐标。

在这个特定的问题中，我们可以看到颜色变化的规律是根据坐标是否为整数（或者说，坐标所在的“棋盘格”是否完整）。因此，我们需要一种方法来将坐标值转化为它们所在的完整棋盘格的位置。floor和ceil函数就是完成这个任务的理想工具。

floor函数会向下取整，也就是返回不大于输入参数的最大整数。这意味着，无论一个数的小数部分是多少，floor函数都会“向下”找到最接近的整数。例如， $\text{floor}(1.6)$ 的结果是1， $\text{floor}(2.0)$ 的结果是2， $\text{floor}(2.1)$ 的结果是2。

而ceil函数则是向上取整，也就是返回不小于输入参数的最小整数。这意味着无论一个数的小数部分是多少，ceil函数都会“向上”找到最接近的整数。例如， $\text{ceil}(1.6)$ 的结果是2， $\text{ceil}(2.0)$ 的结果是2， $\text{ceil}(2.1)$ 的结果是3。

通过使用floor和ceil函数，我们可以将 x_1 和 x_2 坐标转化为它们所在的完整棋盘格的位置，然后根据这个位置来判断该点应该是什么颜色。

为什么要%4:

对于这个问题，我们选择进行取模4（%4）操作是因为棋盘的颜色模式是周期性的，且这个周期为4。

具体来说，如果我们观察棋盘，可以看到每四行或四列，黑白格的颜色模式会重复一次。例如，对于 x_1 ，你会发现第1行和第5行、第2行和第6行、以此类推的颜色模式是一样的。同样的规律也适用于 x_2 ，即第1列和第5列、第2列和第6列等的颜色模式也是一样的。

因此，我们使用取模4操作来识别这个周期性模式。具体地说，对于给定的 x_1 和 x_2 值，我们可以通过' $\text{floor}(x_1) \bmod 4$ '和' $\text{ceil}(x_2) \bmod 4$ '来找出其颜色。这个操作会将 x_1 和 x_2 的值转换为0，1，2或3，这四个值可以对应到棋盘的4种颜色模式。

这就是我们为何在处理 x_1 和 x_2 值时选择进行取模4操作的理由。

(b) 4

这个问题中，我们需要使用形如 $x_1 < c$ 或 $x_2 < c$ 的特征来构建二叉决策树。

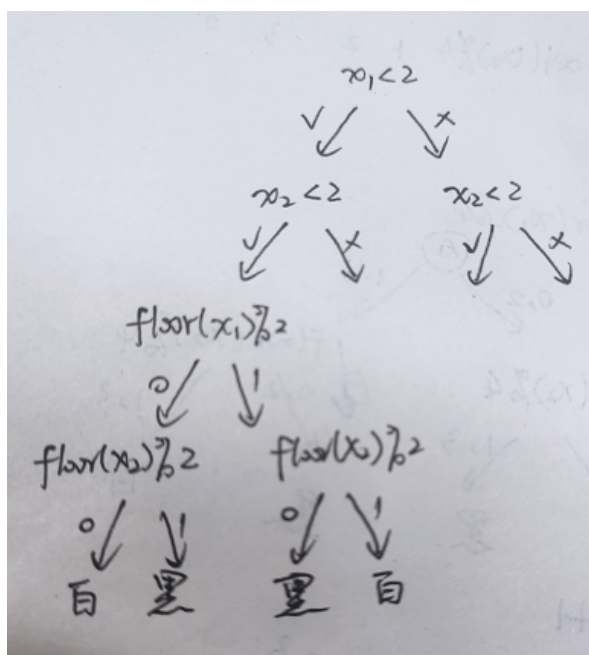
这个问题比较复杂，因为我们需要在每一步都对 x_1 和 x_2 进行检查，以确定阈值 c 。这样，我们就可以将棋盘分割为四个部分，然后再递归地对每一个部分进行处理。

特别需要注意的是，我们需要在每一步都小心地选择阈值 c ，以确保我们可以正确地分类每一个格子。

以下是我们可以采取的步骤：

1. 首先，我们可以选择 $c = 2$ ，然后进行 $x_1 < 2$ 和 $x_2 < 2$ 的检查。这样，我们就可以将棋盘分割为四个部分：左上、右上、左下和右下。
2. 对于左上和右下的部分，我们知道它们都是白色的。所以，我们可以将这两个部分的每一个格子都标记为 -1。
3. 对于右上和左下的部分，我们知道它们都是黑色的。所以，我们可以将这两个部分的每一个格子都标记为 +1。
4. 然后，我们可以递归地对左上和右下的部分进行处理。由于这两个部分都是 2×2 的棋盘，所以我们可以选择 $c = 1$ ，然后再进行 $x_1 < 1$ 和 $x_2 < 1$ 的检查。
5. 重复上述步骤，直到我们处理完所有的格子。

由于在每一步，我们都需要对 x_1 和 x_2 进行检查，所以这个决策树的深度是 $\log_2(4) = 2$ 。但是，由于我们需要对每一个 2×2 的区域进行递归处理，所以实际上，这个决策树的深度是 $2 \times 2 = 4$ 。



(c) 1

方法1:

如果二叉决策树可以使用任何涉及 x_1 和 x_2 的特征，那么它可以直接使用问题给出的颜色规则。颜色规则已经为彩色区域提供了完美的分类。

颜色规则是两个条件的组合， $x_1 < \lfloor x_1 \rfloor + 0.5$ 和 $x_2 < \lceil x_2 \rceil$ ，我们可以将其视为带有复合特征的单一条件。因此，最小深度为1的二叉决策树就足够了。

树的结构可能如下所示：

- 1) 根节点：如果 $x_1 < \lfloor x_1 \rfloor + 0.5$ 异或 $x_2 < \lceil x_2 \rceil$
- 2) 子节点1（真情况）：颜色为黑色
- 3) 子节点2（假情况）：颜色为白色

这里，异或运算符意味着如果 $x_1 < \lfloor x_1 \rfloor + 0.5$ 或 $x_2 < \lceil x_2 \rceil$ 中的任何一个为真，但不是两者都为真，则颜色为黑色。如果两者都为真或两者都为假，那么颜色就是白色。

这棵树根据问题陈述的规则完美地对彩色区域进行了分类。

方法2:

使用涉及 x_1 , x_2 , 或两者的任何特征，我们能构建一个完美分类图1中的颜色区域的二叉决策树，其最小深度为 1。

由于我们可以同时使用 x_1 和 x_2 ，我们可以利用它们的奇偶性来判断一个单元格的白色。观察棋盘，我们可以发现当 x_1 和 x_2 的整数部分都为偶数或者都为奇数时，对应的格子为白色，否则为黑色。

这可以通过以下特征进行分类：

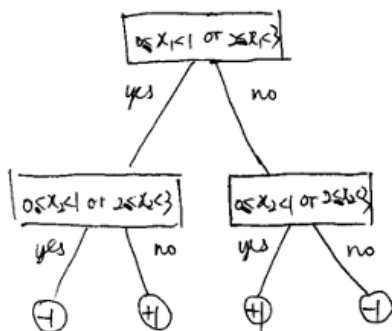
- $f(x_1, x_2) = \text{floor}(x_1) \% 2$ 和 $g(x_1, x_2) = \text{floor}(x_2) \% 2$

然后，我们可以使用以下规则对每一个格子进行分类：

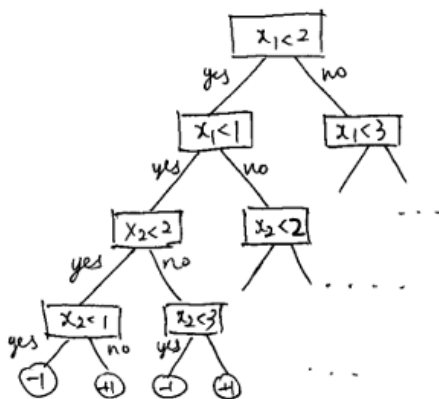
- 如果 $f(x_1, x_2) == g(x_1, x_2)$ ，那么这个格子就是白色的，标记为 -1；
- 否则，这个格子就是黑色的，标记为 +1。

这个规则只涉及到一个决策（比较两个特征是否相等），所以我们的二叉决策树的最小深度是1。

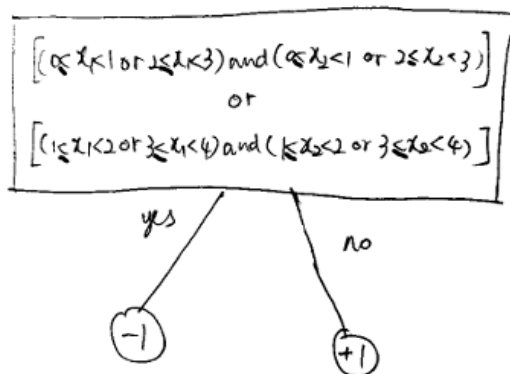
1.(a). 最小深度 = 2



(b). 最小深度 = 4



(c). 最小深度 = 1



2.

(a) C

模型在测试集 D_{test} 上的表现通常略差于它在验证集 D_{val} 上的表现，特别是在剪枝后。这是因为模型是根据验证集进行剪枝的，所以它特别优化以在那个集合上表现良好。虽然这常常提高了泛化性能，但优化不是完美的，模型在未见过的数据上通常表现略差。

因此，正确答案是：

C. 略低于剪枝树在 D_{val} 上的准确率。

这只是一个估计，实际结果可能会因多种因素而变化，包括数据集的具体组成和正在解决的问题的性质。

(b) C

在这三个选项中，对真实误差的最佳近似是：

C. 对应于测试数据 D_{test} 的误差。

解释：训练误差往往低于真实误差，因为模型是在训练数据上进行优化的，可能会过拟合。验证误差用于模型选择和调整超参数，所以也可能低于真实误差。而测试数据集是模型从未见过的数据，其误差通常被视为对真实误差的最好估计。

3.

BC

以下是避免过拟合的有效方法：

B. 为内部节点分裂所需的最小示例数设置阈值。

C. 剪枝树以使交叉验证错误最小化。

解释：

A. 减少训练集大小通常不是一个好主意，因为更少的数据可能会导致模型不能充分学习数据的底层模式，从而导致欠拟合。

B. 为内部节点分裂所需的最小示例数设置阈值是一个常见的策略，可以防止模型在训练数据中的噪声或异常值上过度拟合。

C. 通过剪枝来简化决策树，以使交叉验证错误最小化，也是一种常见的避免过拟合的方法。这种方法通过移除对预测结果影响不大的部分，简化了模型，从而提高了模型在未见过的数据上的预测能力。

D. 增加树的深度通常会导致模型过度拟合训练数据，因为它增加了模型的复杂性。因此，这通常不被认为是避免过拟合的有效方法。

4.

(a) A

是的，可以构建一个决策树，其行为与1-最近邻分类器完全相同。这个决策树将是一个非常细分的决策树，其中每个训练点都会对应一个叶节点。

这样的决策树构建方法如下：

1. 对于每一对相邻的训练点 $x^{(i)}$ 和 $x^{(i+1)}$ ，找到它们的中点 $t^{(i)} = \frac{x^{(i)} + x^{(i+1)}}{2}$ 。这个值将作为决策树在内部节点的切分阈值。
2. 在每个内部节点，决策条件形如 " $x \leq t^{(i)}$ " 或 " $x > t^{(i)}$ "。如果 " $x \leq t^{(i)}$ " 则流向左子节点，否则流向右子节点。
3. 对于每个叶节点，其返回的类标签是对应训练点的类标签。

但是，值得注意的是，尽管这样的决策树在训练集上的表现与1-最近邻分类器相同，但在新的、未见过的测试点上，两者的表现可能会有所不同。此外，这样的决策树可能会非常复杂，并且可能过拟合训练数据。

通过一个例子来更详细地解释如何构建一个决策树来模拟1-最近邻（1-NN）分类器的行为。

给定的训练数据点和标签如下：

数据点 $x^{(i)}$	标签 $y^{(i)}$
1	0
2	1
4	0
6	1
7	0

我们的目标是构建一个决策树，使其对于任何测试点的分类与1-NN分类器相同。

构建决策树过程

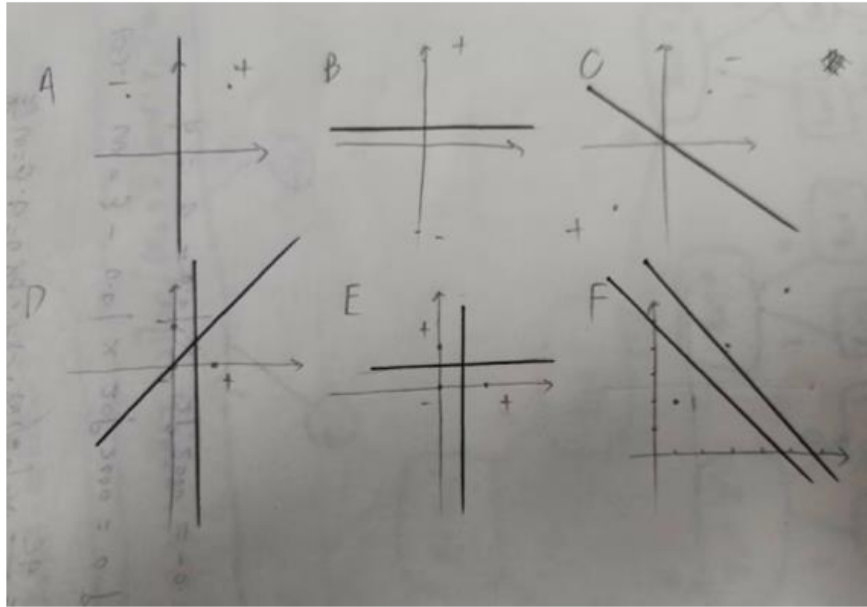
- 根节点：**以第一个数据点 $x^{(1)} = 1$ 作为决策标准。
 - 如果测试点 x 的值 ≤ 1 ，它最近的邻居是 $x^{(1)}$ ，所以预测标签为 0。
 - 如果 $x > 1$ ，移动到下一个节点。
- 第二个节点：**以第二个数据点 $x^{(2)} = 2$ 作为决策标准。
 - 如果 $x \leq 2$ ，它最近的邻居是 $x^{(2)}$ ，所以预测标签为 1。
 - 如果 $x > 2$ ，移动到下一个节点。
- 第三个节点：**以 $x^{(3)} = 4$ 作为决策标准。
 - 如果 $x \leq 4$ ，最近邻是 $x^{(3)}$ ，预测标签为 0。
 - 如果 $x > 4$ ，继续到下一个节点。
- 第四个节点：**以 $x^{(4)} = 6$ 作为决策标准。
 - 如果 $x \leq 6$ ，最近邻是 $x^{(4)}$ ，预测标签为 1。
 - 如果 $x > 6$ ，继续到下一个节点。
- 第五个节点：**以 $x^{(5)} = 7$ 作为最后的决策标准。
 - 如果 $x \leq 7$ ，最近邻是 $x^{(5)}$ ，预测标签为 0。
 - 如果 $x > 7$ ，由于没有更多的训练点，我们可以根据实际情况决定如何设置标签。在实际应用中，这可能需要根据数据的分布或其他规则来确定。

通过上述步骤，我们构建的决策树在每个决策节点上都复现了1-NN分类器的行为。对于任何测试点，该决策树都能找到其最近的训练数据点，并给出与1-NN分类器相同的预测标签。这样，决策树完全模拟了1-NN分类器的行为。

(b)

ABE

$x_j \leq t$ 或 $x_j > t$ 中 t 是常数，所以分割线必须是水平线或竖直线。画出六个选项的分割线如下，图中-代表0标签，+代表1标签，黑色线代表分割线，只有ABE的分割线完全水平或竖直：



所以，正确的答案是A, B, E。

2-k近邻

1. (a) $2/7=0.2857$

因为对于 $k=6$ 偶数的情况下，平局的话那就随机出结果

- (b) BC

如果 $k=12$ 的话，好像会出现平局的情况

2. (a) B

这个陈述是错误的。虽然在训练集上最小化错误率可能在某些情况下是一个好的策略，但这并不总是正确的，特别是当我们考虑到过拟合问题时。如果我们总是选择最小化训练集错误的 k 值，那么我们可能会选择一个过小的 k 值，这可能会导致模型过度复杂，过度适应训练数据的噪声和异常值，而无法很好地泛化到新的、未见过的数据。与此相反，如果我们选择一个过大的 k ，那么模型可能过于简单，无法捕获数据的潜在模式。这就是为什么我们通常会使用交叉验证或者其他形式的模型选择来确定 k 值，而不仅仅是最小化训练集错误。

(b)A. 是的，降低验证错误而不是训练错误更好，因为降低训练错误不会帮助我们的模型泛化，并可能导致过拟合。

(c)B. 不

解释：这不是一个好主意。使用测试集作为选择超参数的验证数据会导致信息泄漏，我们可能会过度优化模型以适应测试数据，这可能导致我们过度估计模型的泛化能力。测试集应该保持独立，只在最后一步用于评估模型的性能。

3. BD 对AC，模型都会无法处理平局情况，不知道如何选取样本点。

4. ABD

解释：

A. 更大的 k 确实会产生更平滑的决策边界，因为分类决策会考虑到更多的近邻样本，从而降低了单个样本带来的影响。

B. 增加 k 的值会降低噪声和离群值的影响，因为在决策过程中使用了更多的样本，从而稀释了这些点的影响。

C. 如果 k 过大，相反，我们可能会导致模型过于简化（欠拟合），而不是过拟合。因为当 k 过大时，模型可能会忽视许多有用的、更接近的邻居，而基于更广泛（可能不太相关）的信息做出决策。

D. 我们确实可以使用交叉验证来帮助确定 k 的值。通过观察不同 k 值下的验证误差，我们可以选择使误差最小化的 k 值。

E. 这个陈述是错误的。我们通常会选择能最小化验证集误差的 k 值，因为这样可以帮助我们找到一个能够更好地泛化到新数据的模型。

5. (a)3

在学生1到6中，谁是学生7的最近邻，使用欧几里德距离？只回答学生ID。

解答：我们可以通过计算学生7与其他每个学生在高中和大学GPA上的欧式距离来找到最近的邻居。经过计算，可以发现学生7的最近邻是学生3。

(b)944

在 k -NN回归中，我们的预测等于最近的 k 个邻居的目标变量（这里是薪资）的平均值。因此，如果 $k = 3$ ，我们需要找到距离学生7最近的3个学生，并计算他们薪资的平均值。本题中最近的是2,3,5号学生

(c)AB

选项A，B都是正确的。这是因为 k -NN算法依赖于计算点之间的距离，而这个距离的计算会受到特征的缩放影响。如果我们改变了特征的尺度（例如，将GPA从4.0缩放到100%），这将影响到计算出的距离，从而可能会改变最近邻居的选择，进而影响到计算的均方误差（MSE）。

对于欧几里德距离来说，如果所有特征同时按相同的比例进行缩放，那么是不会改变样本之间的距离关系，也就不会改变最近邻的选择。在这种情况下，你的理解是正确的。

3-感知机

预备知识：

在线感知器算法是一种用于二元分类器的监督学习算法。它由弗兰克·罗森布拉特在1957年发明，是最早的机器学习算法之一。

感知器算法的工作方式是基于线性预测函数进行预测，该函数将一组权重与特征向量结合起来。在感知器每次犯错误后，权重会逐步学习。

以下是在线感知器算法的基本步骤：

1. 初始化权重向量 θ 和偏置 b 。
2. 对于每一个训练样本 x_i ，执行以下步骤：
 - a. 计算预测标签 $\hat{y}_i = \text{sign}(\theta \cdot x_i + b)$ 。
 - b. 如果预测标签 $\hat{y}_i$ 与真实标签 y_i 不一致，则更新权重向量和偏置，即 $\theta = \theta + y_i x_i$ 和 $b = b + y_i$ 。

这个过程会不断重复，直到感知器在训练集上没有错误，或者达到预设的最大迭代次数。值得注意的是，感知器算法只能在数据是线性可分的情况下收敛。

感知器算法的一个主要优点是它的简单性和易于实现。然而，它的一个主要限制是它无法处理线性不可分的数据。当数据不是线性可分的，感知器算法可能会陷入无限循环。

1.B

该问题涉及到在线感知器算法（online perceptron algorithm）。在线感知器算法是一种迭代学习算法，它在每个时刻只处理一个输入实例，并根据该实例更新模型参数。因此，它的运行结果是依赖于输入实例的顺序的。

如果我们改变了输入实例的顺序（即从 S 变为 S' ），感知器可能会犯不同数量的错误。这是因为在处理每个实例时，感知器都会根据当前的错误来调整其参数。所以，不同的输入顺序可能会导致不同的参数更新路径，从而在训练过程中产生不同数量的错误。

因此，我们不能保证在线感知器算法在 S 和 S' 上犯的错误的数是相同的。所以答案是 "False"。

2.AB

这个问题需要我们确定哪些逻辑函数可以由一个感知器实现。感知器是一种线性分类器，它可以实现一些基本的逻辑函数，例如AND和OR，但不能实现XOR（异或）函数，因为XOR不是线性可分的。

A. AND函数是线性可分的，可以由感知器实现。例如，设置 $w_1 = 1$ ， $w_2 = 1$ ， $b = -1.5$ ，感知器就实现了AND函数。

B. OR函数也是线性可分的，可以由感知器实现。例如，设置 $w_1 = 1$ ， $w_2 = 1$ ， $b = -0.5$ ，感知器就实现了OR函数。

C. XOR（异或）函数是线性不可分的，不能由一个单层感知器实现。XOR函数要求在两个类别的输入间创建一个非线性的决策边界，这是单层感知器无法完成的。

D. 由于A和B都可以被感知器实现，所以"D. None of the above."是错误的。

因此，正确的答案是A和B。

3.C.

感知器算法是一种线性分类器，参数的数量取决于输入特征的数量。在这个问题中，我们的数据集由 N 个样本组成，每个样本有 M 个特征，即 $\mathbf{x}^{(i)} \in \mathbb{R}^M$ 。因此，感知器需要学习的参数数量应与特征数量相等，即 M 个。

这里的参数对应于感知器的权重，每个权重对应一个特征。注意，这个问题假设没有截距项（也称为偏置项）。如果模型中包含截距项，那么需要学习的参数数量将会是 $M + 1$ ，因为截距项也是一个需要学习的参数。

所以，正确答案是C。 M 个参数。

4.B

感知器算法的工作原理是在每次错分类一个实例时，更新模型的权重。如果一个实例被错分类，则我们会将这个实例的特征值乘以它的标签（ y ），然后加到当前的权重上，形成新的权重。特别地，如果初始权重为0，那么最后的权重就是所有被错分类的实例的特征值乘以它们的标签和被错分类的次数之和。

在这个问题中，我们可以根据每个实例被错分类的次数，计算出感知器找到的分隔平面 $\theta = [b, \theta_1, \theta_2, \theta_3]$ 。

具体计算过程如下：

- 对于实例1，有 $10 * (1 * [1, 2, 1, 5]) = [10, 20, 10, 50]$ 。
- 对于实例2，有 $5 * (1 * [1, 5, 3, 3]) = [5, 25, 15, 15]$ 。
- 对于实例3，有 $8 * (1 * [1, 1, 6, 2]) = [8, 8, 48, 16]$ 。
- 对于实例4，有 $2 * (-1 * [1, 7, 2, 1]) = [-2, -14, -4, -2]$ 。
- 对于实例5，有 $3 * (-1 * [1, 3, 2, 6]) = [-3, -9, -6, -18]$ 。

将上述所有实例的结果加起来，我们就得到了感知器找到的分隔平面：

$$\theta = [b, \theta_1, \theta_2, \theta_3] = [10 + 5 + 8 - 2 - 3, 20 + 25 + 8 - 14 - 9, 10 + 15 + 48 - 4 - 6, 50 + 15 + 16 - 2 - 18] = [18, 20, 63, 61]$$

参数更新规则为：

$$\theta = \theta + y_i x_i \text{ 和 } b = b + y_i。$$

所以，正确答案是B。[18, 20, 63, 61]。

5.E

感知器算法的错误界限（mistake bound）是基于数据点到分隔超平面的最小距离和数据点的范数的。

A. 错误。如果分隔超平面到任何数据点的最小距离增加，错误界限实际上会减少，而不是增加。这是因为数据点离分隔超平面越远，感知器分类错误的可能性就越小。

B. 错误。如果整个数据集从原点移动，错误界限不会改变。感知器的错误界限是基于数据点和分隔超平面之间的相对位置，而不是绝对位置。因此，将整个数据集移动并不会改变数据点和分隔超平面之间的相对位置，从而不会影响错误界限。

C. 错误。如果增加数据点之间的距离（即通过某个常数值缩放数据），错误界限实际上会减少。这是因为缩放数据会同时增加数据点的范数和数据点到分隔超平面的距离，而感知器的错误界限与数据点的范数和最小距离的比值成反比。

D. 错误。在讨论感知器算法的错误界限时，应该考虑到它与数据点到分隔超平面的最小距离的平方成反比，而不是仅与最小距离本身成反比。这个界限通常被表述为与最小距离的平方成反比和与数据点的范数的平方成正比。

感知器的错误界限（mistake bound）由以下因素决定：

- 分离超平面与数据点的最小距离的平方 γ^2 （与之成反比）。
- 数据点中的最大范数的平方 R^2 （与之成正比）。

因此，如果最小距离增加，错误界限实际上会减小，因为这意味着数据更容易被分离。这与选项D的表述相矛盾，因此D是错误的。

所以，正确的答案应该是E：“以上都不是”。

补充知识：the mistake bound of the perceptron algorithm

感知器算法的错误界限（mistake bound）是一种理论界限，用于衡量在二元分类问题中，感知器算法在最坏情况下可能会犯多少错误。

这个界限是基于数据的一些性质，包括数据点和分隔超平面之间的几何距离，以及数据点的范数（即向量的长度）。

具体来说，假设我们有一个线性可分的数据集，其中正例和负例可以被一个超平面完美地分开。进一步假设 γ 是数据点到这个超平面的最小距离，而 R 是数据点的最大范数。在这种情况下，感知器算法的错误界限可以被证明是 $(R/\gamma)^2$ 。这意味着在最坏的情况下，感知器算法可能需要 $(R/\gamma)^2$ 步才能找到一个将所有正例和负例完美分开的超平面。

这个界限是一个理论上的最大值，实际中的错误次数可能会少一些。然而，这个界限提供了对感知器算法性能的一种保证。

6. C

在感知器算法中，每次更新权重向量 θ 时，都是根据被错分类的样本 x 及其标签 y 来更新的，即 $\theta = \theta + yx$ 。由于我们的数据满足 $x_1 - x_2 = 0$ ，这意味着 $x_1 = x_2$ 。

因此，对于任何被错分类的样本，我们都有 $x = [x_1, x_2] = [x_1, x_1]$ ，这将导致权重向量 θ 的两个分量始终保持相等。换句话说，无论如何更新， θ 的两个分量始终是相同的。

因此，任何不满足 $\theta_1 = \theta_2$ 的 θ 值都不会在感知器算法中出现。从给定的选项中，我们可以看到，只有选项C $[-3, 0]$ 和选项D $[-6, -4]$ 不满足这个条件，因此，它们都是 θ 在运行感知器算法时不会取到的值。

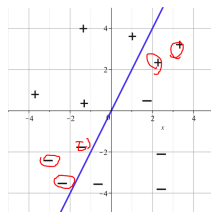
所以，这个问题的答案为C $[-3, 0]$ 。

$\theta = \theta + y \cdot [x_1, x_2]$, 这里的 $x_1 = x_2$

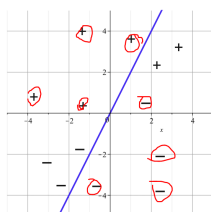
对于D, $y=-1$ 即可，对于C，则答案应该是 $[-3, -1]$

7. AB

A.



B.



8.

这道题目涉及到了计算感知器错误界限（Perceptron Mistake Bound）的各个要素，包括数据点的范围（R）和数据点到分隔超平面的最小距离（margin），以及用这些要素来计算错误界限。

(a) 3.7417

计算数据点的范围 R ：这个范围是指原点到数据点的最大欧几里得距离。这个距离可以通过计算每个数据点到原点的距离，然后取最大值得到。数据点到原点的距离可以通过计算以下公式得到： $\sqrt{x_0^2 + x_1^2 + x_2^2}$ 。在这个例子中，所有的数据点的 x_0 都是1，所以我们只需要计算和比较以下的值： $\sqrt{1^2 + (-2)^2 + 2^2}$ ， $\sqrt{1^2 + (-1)^2 + (-3)^2}$ ， $\sqrt{1^2 + (-2)^2 + (-3)^2}$ ， $\sqrt{1^2 + 0^2 + 1^2}$ 和 $\sqrt{1^2 + 2^2 + (-1)^2}$ 。取这些值中的最大值即为 $R = \sqrt{14} = 3.7417$ 。

(b) 1.024

计算数据集的 margin：这个 margin 是指数据点到分隔超平面的最小距离。这个距离可以通过计算每个数据点到超平面的距离，然后取最小值得到。数据点到超平面的距离可以通过计算以下公式得到： $|\theta^{*T} x| / \|\theta^*\|$ ，其中 x 是数据点， θ^* 是超平面的法向量。在这个例子中，所有的数据点的 x_0 都是1，所以我们只需要计算和比较以下的值：

$|6 * 1 + 3 * -2 + 4 * 2| / \|\theta^*\|$ ， $|6 * 1 + 3 * -1 + 4 * -3| / \|\theta^*\|$ ， $|6 * 1 + 3 * -2 + 4 * -3| / \|\theta^*\|$ ， $|6 * 1 + 3 * 0 + 4 * 1| / \|\theta^*\|$ 和 $|6 * 1 + 3 * 2 + 4 * -1| / \|\theta^*\|$ 。取这些值中的最小值即为 $margin = 8 / \sqrt{6^2 + 3^2 + 4^2} = 1.024$ 。

(c)

计算感知器错误界限：根据感知器错误界限的公式，错误界限等于 $(R / margin)^2$ 。在这个例子中，只需要将在上述两步中计算得到的 R 和 margin 值代入这个公式就可以得到错误界限。

$$\left(\frac{\sqrt{14}}{8/\sqrt{61}}\right)^2 = 13.344$$

4-线性回归

1.

这个问题涉及到了使用梯度下降法来优化线性回归模型的参数。我们的目标函数是 $\frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$ ，其中 N 是数据点的数量， w 是权重， b 是截距。

给定的数据集是：

x	9.0	2.0	6.0	1.0	8.0
y	1.0	0.0	3.0	0.0	1.0

(a) 首先，我们需要计算损失函数相对于权重 w 的梯度。这个梯度可以通过以下公式计算：

$$\frac{\partial}{\partial w} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2 = \frac{2}{N} \sum_{i=1}^N x_i (wx_i + b - y_i) = 209.2000$$

我们可以将给定的数据和初始的 w 和 b 值代入这个公式，以计算梯度。

(b) 接下来，我们需要计算损失函数相对于截距 b 的梯度。这个梯度可以通过以下公式计算：

$$\frac{\partial}{\partial b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2 = \frac{2}{N} \sum_{i=1}^N (wx_i + b - y_i) = 29.2000$$

我们可以将给定的数据和初始的 w 和 b 值代入这个公式，以计算梯度。

(c) 最后，我们需要执行一步梯度下降的更新。这可以通过以下公式完成：

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial}{\partial w} \text{Loss}(w, b) = 3 - 0.01 \times 209.2 = 0.9080$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial}{\partial b} \text{Loss}(w, b) = 0 - 0.01 \times 29.2 = -0.2920$$

其中 η 是学习率。我们可以将我们在上面计算的梯度和给定的学习率 $\eta = 0.01$ 代入这个公式，以计算新的 w 和 b 值。

2.

(a) C

答案是 C. $w_1 = w_2, b_1 \neq b_2$ 。这是因为，在线性回归中，斜率（即 w_1 或 w_2 ）由输入（ x ）和输出（ y ）之间的关系决定，而偏置项（即 b_1 或 b_2 ）主要决定了直线的垂直位置。当我们将所有的 x 和 y 都加上一个常数后，这并不会改变 x 和 y 之间的关系，所以 w_1 应该等于 w_2 。但是，这种加法操作会改变直线的垂直位置，所以 b_1 不应该等于 b_2 。

(b) 参考答案

答案1：这是因为重复数据集中的部分行，实际上改变了数据集的分布。具体来说，被复制的数据点在计算均方误差时，会有更大的权重。因此，线性回归模型会更倾向于适应这些被复制的数据点，从而可能导致学到的参数与原始未复制数据集上学到的参数不同。

答案2：

除非数据完美地符合最佳拟合线，否则参数可能会有所不同。当我们进行梯度下降时，不在线上的点将创建一个非零的梯度，反过来，这将改变参数 w_1 和 b_1 。另一方面，如果被复制的点在最佳拟合线上，梯度将为 0，学习到的参数将不会有所不同。

这位同学的回答是正确的。他/她用了一种不同的方式来解释同样的概念。在梯度下降的过程中，离最佳拟合线较远的点会产生更大的梯度，因此对参数更新的影响也更大。如果被复制的点恰好在最佳拟合线上，那么这些点对梯度的贡献为零，复制这些点并不会改变学习到的参数。但如果被复制的点不在最佳拟合线上，那么复制这些点会使它们对梯度的总贡献增加，从而可能改变学习到的参数。

3.

(a) 在这种设置中，目标函数 $J(\theta)$ 是所有点的平均 log-cosh 损失，所以

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \log(\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)}))$$

(b) $J^{(i)}(\theta)$ 关于 θ_j 的偏导数为

$$\frac{\partial J^{(i)}}{\partial \theta_j} = \frac{\sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)})}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} \cdot x_j^{(i)} = \tanh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$$

(c) $J^{(i)}(\theta)$ 关于 θ 的梯度为

$$\nabla J^{(i)}(\theta) = \left[\frac{\partial J^{(i)}}{\partial \theta_1}, \frac{\partial J^{(i)}}{\partial \theta_2}, \dots, \frac{\partial J^{(i)}}{\partial \theta_k} \right]^T$$

$$\begin{aligned} \frac{\partial J^{(i)}}{\partial \theta} &= \frac{1}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} * \sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * x^{(i)} \\ \frac{\partial J^{(i)}}{\partial \theta} &= \frac{1}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} * \sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_k^{(i)}] \\ \frac{\partial J^{(i)}}{\partial \theta} &= \tanh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_k^{(i)}] \end{aligned}$$

(d) [参考答案] 执行一次迭代的梯度下降的伪代码为：

```
lr = 0.1
theta = zero(x.shape[1])
gradient = zeros(x.shape)

for i in range(x.shape[0]):
    xi = x[i]
    yi = y[i]
    gradient[i] = xi * tanh(dot(theta, xi) - yi)
theta = theta - lr * mean(gradient, axis=0)
```

1. 首先，设置学习率 `lr` 为 0.1，参数向量 `theta` 初始化为与数据特征维度相同长度的零向量。
2. 然后，初始化一个形状和 `x` 相同的零矩阵 `gradient`，用于保存每个样本的梯度。
3. 在主循环中，对每个样本 `x[i]` 和对应的目标值 `y[i]`，计算梯度，这一步实现了公式 $\tanh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) \cdot \mathbf{x}_j^{(i)}$ 。
4. 最后，计算所有样本梯度的平均值，并用此值更新参数向量 `theta`，这一步实现了梯度下降的参数更新步骤。

伪代码正确地实现了一次梯度下降的迭代过程。需要注意的是，这只是一次迭代，通常我们需要多次迭代（例如设定一个最大迭代次数或者直到满足某种停止准则）才能找到损失函数的最小值。

简洁答案(批改版)

1

1. (1) 2 【3分】
(2) 4 【3分】
(3) 1 【3分】
2. (a) C 【2分】
(B) C 【2分】
3. BC 【2分】
4. (a) A 【4分】
(b) ABE 【3分】

2 k近邻

1. (a) $\frac{2}{7} = 0.2857$ 【2分】
(b) BC 【3分】
2. (a) B 【2分】
(b) A 【2分】
(c) B 【4分】
3. BD 【2分】
4. ABD 【4分】
5. (a) 3 【2分】

(b) 944 【4分】

(c) AB 【2分】

3. 感知机

1. B 【2分】

2. AB 【3分】

3. C 【2分】

4. B 【2分】

5. E 【2分】

6. C 【2分】

7. AB 【3分】

8. (a) 3.7417 【4分】

(b) 1.0243 【4分】

(c) 13.3438 【2分】

4 线性回归

1. (a) 209.2 【4分】

(b) 29.2 【3分】

(c) 0.908 -0.292 【3分】

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial}{\partial w} \text{Loss}(w, b) = 3 - 0.01 \times 209.2 = 0.9080$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial}{\partial b} \text{Loss}(w, b) = 0 - 0.01 \times 29.2 = -0.2920$$

2. (a) C 【2分】

(b) 【2分】

这是因为重复数据集中的部分行，实际上改变了数据集的分布。具体来说，被复制的数据点在计算均方误差时，会有更大的权重。因此，线性回归模型会更倾向于适应这些被复制的数据点，从而可能导致学到的参数与原始未复制数据集上学到的参数不同。

3.

(a) 【2分】在这种设置中，目标函数 $J(\theta)$ 是所有点的平均 log-cosh 损失，所以

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \log(\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)}))$$

(b) 【3分】 $J^{(i)}(\theta)$ 关于 θ_j 的偏导数为

$$\frac{\partial J^{(i)}}{\partial \theta_j} = \frac{\sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)})}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} \cdot x_j^{(i)} = \tanh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$$

(c) 【2分】 $J^{(i)}(\theta)$ 关于 θ 的梯度为

$$\nabla J^{(i)}(\theta) = \left[\frac{\partial J^{(i)}}{\partial \theta_1}, \frac{\partial J^{(i)}}{\partial \theta_2}, \dots, \frac{\partial J^{(i)}}{\partial \theta_k} \right]^T$$

$$\frac{\partial J^{(i)}}{\partial \theta} = \frac{1}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} * \sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * \mathbf{x}^{(i)}$$

$$\frac{\partial J^{(i)}}{\partial \theta} = \frac{1}{\cosh(\theta^T \mathbf{x}^{(i)} - y^{(i)})} * \sinh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_k^{(i)}]$$

$$\frac{\partial J^{(i)}}{\partial \theta} = \tanh(\theta^T \mathbf{x}^{(i)} - y^{(i)}) * [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, \dots, x_k^{(i)}]$$

(d) 【4分】 执行一次迭代的梯度下降的伪代码为：

```
alpha = 0.1
theta = zero(x.shape[1])
gradient = zeros(x.shape)

for i in range(x.shape[0]):
    xi = x[i]
    yi = y[i]
    gradient[i] = xi * tanh(dot(theta, xi) - yi)
theta = theta - alpha * mean(gradient, axis=0)
```

1. 首先，设置学习率 `alpha` 为 0.1，参数向量 `theta` 初始化为与数据特征维度相同长度的零向量。
2. 然后，初始化一个形状和 `x` 相同的零矩阵 `gradient`，用于保存每个样本的梯度。
3. 在主循环中，对每个样本 `x[i]` 和对应的目标值 `y[i]`，计算梯度，这一步实现了公式 $\tanh(\theta^T x^{(i)} - y^{(i)}) \cdot x_j^{(i)}$ 。
4. 最后，计算所有样本梯度的平均值，并用此值更新参数向量 `theta`，这一步实现了梯度下降的参数更新步骤。

伪代码正确地实现了一次梯度下降的迭代过程。需要注意的是，这只是一次迭代，通常我们需要多次迭代（例如设定一个最大迭代次数或者直到满足某种停止准则）才能找到损失函数的最小值。